

# **CARTFORGE – JENKINS CONTINUOUS INTEGRATION ENVIRONMENT**

## **Project Report**

### **Submitted By**

**Shubham Khade**

### **Course**

**DevOps and AWS**

### **Institute**

**IT Vedant**

---

## **1. Project Overview**

The CartForge Jenkins Continuous Integration Environment is a DevOps project developed to understand and implement Continuous Integration using Jenkins.

The main purpose of this project is to automate the software development workflow. Jenkins was integrated with GitHub to automatically build, test, package, and deliver the application artifact.

The project includes Jenkins installation and configuration, GitHub integration, Freestyle Jobs, Jenkins Pipeline, Jenkins Agent configuration, GitHub Webhooks, artifact management, and pipeline monitoring.

The Jenkins environment was implemented using Ubuntu EC2 instances on AWS.

---

## **2. Project Objective**

The objective of this project was to gain practical experience with Jenkins and Continuous Integration.

The main objectives were:

- Install and configure Jenkins on Ubuntu Linux.
- Integrate Jenkins with GitHub.
- Create a Jenkins Freestyle Job.
- Create a Jenkins Pipeline using a Jenkinsfile.
- Configure a Jenkins Agent.
- Implement distributed builds.

- Configure GitHub Webhooks.
  - Automate pipeline execution.
  - Generate and deliver application artifacts.
  - Monitor Jenkins Pipeline execution.
- 

### **3. Technologies Used**

The following technologies and tools were used in this project:

- AWS EC2
  - Ubuntu Linux
  - Jenkins
  - Java
  - Git
  - GitHub
  - Node.js
  - npm
  - SSH
  - GitHub Webhooks
- 

### **4. Jenkins Architecture**

The project architecture consists of the following components:

#### **GitHub Repository**

The GitHub repository stores the application source code and Jenkinsfile.

#### **Jenkins Controller**

The Jenkins Controller manages Jenkins jobs, pipelines, configurations, and build execution.

#### **Jenkins Agent**

The Jenkins Agent is configured as a separate node and is responsible for executing pipeline jobs.

#### **GitHub Webhook**

The GitHub Webhook automatically notifies Jenkins whenever changes are pushed to the repository.

## **Jenkins Pipeline**

The Jenkins Pipeline automates the application build, testing, packaging, and artifact delivery process.

---

### **Architecture Flow**

Developer



GitHub Repository



GitHub Webhook



Jenkins Controller



Jenkins Agent



Build and Test



Package Application



Deliver Artifact

---

## **5. Jenkins Installation**

Jenkins was installed on an Ubuntu EC2 instance.

First, the Ubuntu system was updated.

```
sudo apt update
```

```
sudo apt upgrade -y
```

Java was installed because Jenkins requires Java to run.

```
sudo apt install fontconfig openjdk-21-jre -y
```

After installing Java, Jenkins was installed and configured.

The Jenkins service was started and enabled.

```
sudo systemctl start jenkins
```

```
sudo systemctl enable jenkins
```

The Jenkins Dashboard was accessed using:

```
http://JENKINS-SERVER-PUBLIC-IP:8080
```

The initial Jenkins setup was completed, suggested plugins were installed, and an Administrator account was created.

---

## **6. Jenkins CI Environment Configuration**

The Jenkins environment was configured with the required development tools.

Git was installed for source code management.

```
sudo apt install git -y
```

Node.js and npm were installed to support the application build process.

The installations were verified using:

```
git --version
```

```
node -v
```

```
npm -v
```

---

## **7. Jenkins Freestyle Job**

A Jenkins Freestyle Job was created to understand the basic Jenkins build process.

The Freestyle Job was connected with the GitHub repository.

The job was manually executed, and the Console Output was monitored.

The successful execution of the Freestyle Job confirmed that Jenkins was correctly configured to interact with the GitHub repository.

---

## **8. Jenkins Agent Configuration**

A separate Ubuntu EC2 instance was configured as a Jenkins Agent.

The Agent was connected to the Jenkins Controller using SSH.

The Agent configuration included:

Name:

CartForge-Agent

Remote Root Directory:

/home/ubuntu/jenkins

Label:

cartforge-agent

Launch Method:

Launch agents via SSH

The Jenkins Pipeline was configured to execute on the Jenkins Agent using the following label:

cartforge-agent

This configuration helped demonstrate distributed builds, where the Jenkins Controller manages the pipeline while the Jenkins Agent executes the build process.

---

## 9. Jenkins Pipeline

A Jenkins Pipeline was created using a Jenkinsfile.

The pipeline was designed to automate the following stages:

### Stage 1: Clone Source Code

The application source code is downloaded from the GitHub repository.

### Stage 2: Install Dependencies

The required Node.js dependencies are installed using npm.

```
npm install
```

### Stage 3: Build Application

The application build process is executed.

npm run build

#### **Stage 4: Run Tests**

Application tests are executed.

npm test

#### **Stage 5: Package Application**

The application files are packaged into a compressed artifact.

#### **Stage 6: Deliver Artifact**

The generated artifact is stored in Jenkins.

---

### **10. Pipeline Workflow**

The complete pipeline workflow is:

Clone Source Code



Install Dependencies



Build Application



Run Tests



Package Application



Deliver Artifact

This workflow demonstrates how Jenkins automates the Continuous Integration process.

---

### **11. GitHub Webhook Integration**

GitHub Webhook integration was configured to automate Jenkins builds.

Whenever a developer makes changes to the application and pushes the changes to GitHub, GitHub automatically sends a webhook request to Jenkins.

Jenkins receives the webhook request and automatically triggers the pipeline.

The workflow is:

Developer Changes Code



Git Add



Git Commit



Git Push



GitHub Webhook



Jenkins Pipeline Triggered



Build and Test



Artifact Generated

This demonstrates Continuous Integration automation.

---

## 12. Challenges Faced

During the implementation of this project, several challenges were faced.

### Jenkinsfile Configuration

Initially, Jenkins was unable to locate the Jenkinsfile from the GitHub repository.

This issue was resolved by placing the Jenkinsfile in the correct repository location and configuring the correct branch and Script Path.

### Jenkins Agent Configuration

Configuring the Jenkins Agent required proper SSH connectivity, credentials, Java installation, and Remote Root Directory configuration.

### GitHub Webhook Configuration

The webhook required the correct Jenkins URL and proper network configuration to allow GitHub to communicate with Jenkins.

### **Pipeline Troubleshooting**

Some issues were related to application dependencies, directory paths, and Node.js commands.

These challenges improved my troubleshooting and problem-solving skills.

---

### **13. Industry-Level Importance**

This project helped me understand how Continuous Integration works in a real industry environment.

In software companies, multiple developers continuously make changes to application code.

Instead of manually building and testing the application, Jenkins can automatically perform these tasks whenever code is pushed to the repository.

This provides several benefits:

- Reduces manual work.
- Detects errors faster.
- Improves software quality.
- Provides faster feedback to developers.
- Automates repetitive tasks.
- Supports Continuous Integration and Continuous Delivery.

The Jenkins Agent architecture also demonstrates how large organizations distribute build workloads across multiple systems.

---

### **14. Best Part of the Project**

The best part of this project was understanding the complete automation workflow.

It was interesting to see how a simple code push to GitHub can automatically trigger Jenkins and start the complete pipeline.

The complete workflow was:

Code Push



GitHub Webhook



Jenkins Trigger



Pipeline Execution



Build



Test



Package



Artifact Delivery

This provided practical experience with a real-world DevOps Continuous Integration workflow.

---

## 15. Learning Outcomes

Through this project, I gained practical knowledge of:

- Jenkins installation and configuration.
- Jenkins administration.
- Git and GitHub integration.
- Jenkins Freestyle Jobs.
- Jenkins Pipeline.
- Pipeline as Code using Jenkinsfile.
- Jenkins Agents.
- Distributed builds.
- SSH connectivity.
- GitHub Webhooks.

- Continuous Integration.
  - Artifact generation.
  - Pipeline monitoring.
  - Console Output analysis.
  - DevOps troubleshooting.
- 

## **16. Conclusion**

The CartForge Jenkins Continuous Integration Environment was successfully implemented to demonstrate Continuous Integration using Jenkins.

The project integrated GitHub, Jenkins Controller, Jenkins Agent, Jenkins Pipeline, and GitHub Webhooks into a single automated workflow.

This project provided practical experience with important DevOps concepts and tools used in real-world software development environments.

Through this implementation, I gained a better understanding of how Jenkins automates software builds, testing, packaging, and artifact delivery.

This project has helped strengthen my foundation in Jenkins and Continuous Integration and will be useful for future DevOps and Cloud projects.

---

## **17. Screenshots**

### **Jenkins Dashboard**

**Insert Jenkins Dashboard Screenshot Here**

---

### **Jenkins Freestyle Job**

**Insert Freestyle Job Screenshot Here**

---

### **Jenkins Pipeline Stage View**

**Insert Pipeline Stage View Screenshot Here**

---

### **Jenkins Agent Online**

**Insert Jenkins Agent Screenshot Here**

---

**GitHub Webhook Trigger**

**Insert GitHub Webhook Screenshot Here**

---

**Submitted By**

**Shubham Khade**

**DevOps and AWS Student**

**IT Vedant**